



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

TÍTULO DE LA MEMORIA/TESIS

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERE CIVIL EN COMPUTACIÓN

NOMBRE NOMBRE APELLIDO APELLIDO

PROFESOR GUÍA:
NOMBRE APELLIDO APELLIDO

PROFESORA CO-GUÍA:
NOMBRE APELLIDO APELLIDO

MIEMBROS DE LA COMISIÓN:
NOMBRE COMPLETO UNO
NOMBRE COMPLETO DOS
NOMBRE COMPLETO TRES

SANTIAGO DE CHILE
2026

RESUMEN DE LA MEMORIA PARA OPTAR AL
TÍTULO DE INGENIERE CIVIL EN COMPUTACIÓN
POR: NOMBRE NOMBRE APELLIDO APELLIDO
FECHA: 2026
PROF. GUIA: NOMBRE APELLIDO APELLIDO

TÍTULO DE LA MEMORIA/TESIS

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Una dedicatoria especial para alguien especial.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Tabla de Contenido

1. Introducción	1
1.1. Problema abordado	1
1.2. Solución propuesta	1
1.3. Objetivos	2
1.4. Metodología	2
2. Trabajo relacionado	3
3. Diseño del videojuego	4
3.1. Género y mecánicas principales	4
3.2. Diseño de niveles y entornos de prueba	4
3.3. Diseño y arquetipos de enemigos	5
4. Sistema adaptativo basado en el comportamiento del jugador	6
5. Implementación del videojuego	7
5.1. Estadísticas del jugador	9
5.2. Componente de combate	10
5.3. Sistema de input	11
5.4. Ataque melee del jugador	12
5.5. proyectiles	13
5.6. Hechizos y consumibles del jugador	14
5.7. Esquiva	16
5.8. Fijado de objetivo (Target Lock)	17
6. Prueba de concepto y evaluación	19
7. Conclusión y trabajo futuro	20

Bibliografía	21
A. Anexo	22

Índice de Tablas

Índice de Ilustraciones

Capítulo 1

Introducción

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

1.1. Problema abordado

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

1.2. Solución propuesta

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem,

ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

1.3. Objetivos

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1.4. Metodología

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et.

Capítulo 2

Trabajo relacionado

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Capítulo 3

Diseño del videojuego

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

3.1. Género y mecánicas principales

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum.

3.2. Diseño de niveles y entornos de prueba

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis

philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

3.3. Diseño y arquetipos de enemigos

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

asdadaasdas

Capítulo 4

Sistema adaptativo basado en el comportamiento del jugador

Capítulo 5

Implementación del videojuego

Para el proyecto se decidió utilizar Unreal Engine 5.6. El juego se construyó tomando como base la plantilla *Third Person* incluida en Unreal Engine. Esta corresponde a un ejemplo diseñado para demostrar las funcionalidades fundamentales del sistema de personajes en tres dimensiones, incorporando un escenario simple, una cámara orbital y un personaje capaz de desplazarse, correr y saltar. Si bien a primera vista los elementos presentados parecen utilizables directamente para la implementación del proyecto, la plantilla posee diversos aspectos que difieren del objetivo buscado y que requieren modificaciones específicas.

La plantilla *Third Person* proporciona una estructura de proyecto preconfigurada que incluye varios componentes clave. El elemento principal es el `BP_ThirdPersonCharacter`, un Blueprint que representa al personaje jugable y contiene toda la lógica de movimiento, entrada de usuario y comportamiento básico.

A lo largo de esta sección se hará referencia frecuente al concepto de Blueprint. Los Blueprints son el sistema de *scripting* visual de Unreal Engine que permite crear lógica de juego mediante nodos interconectados, sin necesidad de escribir código tradicional. Esto facilita la iteración rápida y permite a diseñadores y artistas contribuir directamente a la programación del juego.

Este Blueprint incluye componentes como la cápsula de colisión, la malla del personaje (*Skeletal Mesh*), el componente de movimiento (*Character Movement Component*) y la cámara con su brazo de resorte (*Spring Arm*). Adicionalmente, la plantilla incluye el *Game Mode*, que define las reglas básicas del juego, y diversos assets como animaciones base, materiales y el escenario de demostración.

Un ejemplo de los ajustes necesarios es la estructura del personaje y sus sistemas de locomoción, los cuales, si bien sirven como referencia inicial, deben ser adaptados o reemplazados según las necesidades mecánicas del juego. Del mismo modo, el comportamiento de la cámara, las animaciones base y la lógica de entrada proporcionadas por defecto funcionan como punto de partida, pero no necesariamente representan la experiencia final deseada.

Por ejemplo, se tuvieron que cambiar las animaciones de movimiento del personaje por algunas que soportaran *Strafe Movement*. El *Strafe Movement* (o simplemente *strafing*) consiste en moverse lateralmente sin perder la orientación hacia un objetivo específico. En lugar de girar todo el cuerpo para caminar hacia la izquierda o la derecha, el personaje desliza su movimiento lateral mientras mantiene la mirada fija hacia adelante o hacia un blanco. Este tipo de movimiento es común en juegos de acción y combate, ya que permite al jugador mantener contacto visual con enemigos mientras se desliza en cualquier dirección.

Se importaron animaciones que soportan el *Strafe Movement*, con las opciones de *Root Motion* y *Enable Root Motion* habilitadas. El *Root Motion* permite que el movimiento del personaje sea controlado directamente por las animaciones en lugar de ser aplicado mediante código, lo que resulta en desplazamientos más naturales y sincronizados con las animaciones reproducidas.

Lo siguiente fue crear un *Blend Space* asociado al esqueleto del jugador, para poder transicionar animaciones dependiendo de la velocidad y dirección del jugador. Un *Blend Space* es un asset de Unreal Engine que permite interpolar suavemente entre múltiples animaciones basándose en valores de entrada. En este caso, se configuró con dos ejes: uno horizontal que representa la dirección lateral del movimiento y uno vertical que representa la velocidad hacia adelante o atrás.

En el *Animation Blueprint* del personaje (*ABP_Manny*), específicamente en el *Animation Graph* dentro de la máquina de estados de *Locomotion*, se reemplazó la animación de *Idle* que venía en el template por una propia, perteneciente al conjunto de animaciones del *Strafe Movement*.

Luego, se reemplazó en el estado de *Walking/Run* el *Blend Space* que venía en el template por el *Blend Space* creado previamente, permitiendo así transiciones fluidas entre las diferentes direcciones de movimiento mientras se mantiene la orientación del personaje. También se desactivó el parámetro «Orient Rotation to Movement» y se activó el parámetro «Use Controller Desired Rotation» para que el jugador mire siempre hacia la dirección que indica la cámara, independientemente de la dirección en la que se mueva.

Cabe recalcar que la mayoría del desarrollo de las implementaciones está realizado en Blueprint. Sin embargo, en el futuro se espera incorporar componentes heredados de C++.

Para desarrollar las capacidades de combate del jugador, se decidió crear un componente `BPC_Combat` que hereda de *Actor Component* y se ancla al Blueprint del jugador, de tal forma que esté centralizada la lógica de combate. De igual forma, para las estadísticas del jugador se creó y ancló un componente `BPC_Stats`.

5.1. Estadísticas del jugador

Como se mencionó anteriormente, las estadísticas del jugador se implementaron en el componente `BPC_Stats` anclado al jugador. Este componente posee múltiples variables, entre las cuales están:

1. `health`: Tipo *Float*. Representa la vida actual del jugador.
2. `Progress Bar Ref`: Tipo *Progress Bar*. Es la referencia a la barra de progreso que muestra la vida.
3. `maxHealth`: Tipo *Float*. Representa el valor máximo de vida del jugador.
4. `stamina`: Tipo *Float*. Representa la stamina actual del jugador.
5. `maxStamina`: Tipo *Float*. Representa la stamina máxima del jugador.
6. `Stam Bar Ref`: Tipo *WB Player Stamina*. Es la referencia al widget que muestra la barra de stamina.
7. `canRecoverStamina`: Tipo *Boolean*. Indica si el jugador puede recuperar stamina.
8. `maxMana`: Tipo *Float*. Representa el valor máximo de maná del jugador.
9. `mana`: Tipo *Float*. Representa el maná actual del jugador.
10. `Mana Bar Ref`: Tipo *Progress Bar*. Es la referencia a la barra que muestra el maná.

La funcionalidad principal de este componente es una función llamada `IncreaseVal`, la cual recibe como parámetros un enum que indica la estadística que se quiere modificar, junto con el valor del cambio. Si la estadística a modificar es la vida (`health`), se disminuye la variable de vida por el valor deseado. Luego se calcula el porcentaje de vida actual (`health/maxHealth`), para finalmente, desde la referencia a la barra (`Progress Bar Ref`), llamar al método `Set Percent`, que se encarga de actualizar visualmente la barra con el porcentaje calculado previamente. En caso de recibir el enum perteneciente a la estadística de maná, el proceso es análogo. Para la estadística de stamina también es similar, con la diferencia de que el Widget de la stamina no se muestra siempre, solo cuando se está gastando o recuperando stamina, similar a como lo implementan juegos como *The Legend of Zelda: Breath of the Wild*¹.

Así, al recibirse el enum de stamina, se debe revisar si existe un widget creado en pantalla. Si se posee, simplemente se le indica al Widget que cambie su porcentaje y al final se revisa si el jugador posee toda la stamina. En caso contrario, significa que el widget no ha sido creado, por lo que se crea y se añade a la pantalla.

¹The Legend of Zelda: Breath of the Wild es un videojuego de acción y aventuras en mundo abierto desarrollado y publicado por Nintendo en 2017. El juego introduce mecánicas innovadoras de supervivencia y exploración, incluyendo un sistema de stamina que se muestra dinámicamente solo cuando se está utilizando o recuperando. Disponible en sitio web.

Finalmente, se revisa si la stamina es mayor o igual al máximo (100%). En caso de ser verdadero, significa que el jugador ya posee toda la stamina, por lo que se procede a remover el widget de la pantalla.

Al iniciarse el componente este activa dos timers que se ejecutan cada 0.01 segundos y llaman a los eventos `RegainStamina` y `RegainMana`. Estos eventos pasan por una rama que verifica si corresponde regenerar la estadística respectiva; si la condición es verdadera, se usa el nodo `IncreaseVal` para aumentar la stamina o el maná en pequeñas cantidades, creando así un sistema de regeneración continua controlada por condiciones. Es importante mencionar que este enfoque de regeneración mediante timers de alta frecuencia podría optimizarse en el futuro.

5.2. Componente de combate

Como se mencionó anteriormente, se creó un *Actor Component* llamado `BPC_Combat` que se ancla al jugador y centraliza toda la lógica relacionada con el combate.

Este componente posee múltiples variables, entre las cuales están:

1. `TargetLock`: Tipo *Actor*. Es la referencia al objetivo que se tiene fijado por la cámara.
2. `Player BP`: Tipo *Object Reference* a *BP Third Person Character*. Referencia al jugador.
3. `TargetLockWidget`: Tipo *Object Reference* a *BP Target Lock Widget*. Es la referencia al widget del símbolo que se crea cuando se fija un objetivo.
4. `isAttacking`: Tipo *Boolean*. Es una bandera que se levanta cuando el jugador está atacando.
5. `isCombo`: Tipo *Boolean*. Bandera que se levanta cuando el jugador tiene la intención de continuar el ataque.
6. `isDodging`: Tipo *Boolean*. Bandera que se levanta cuando el jugador está realizando una esquivas.
7. `StamWidget`: *Object Reference* a *WB Player Stamina*. Es la referencia al widget de la stamina que se muestra cuando se gasta o recupera stamina.
8. `StaminaRollUse`: Tipo *Float*. Define la cantidad de stamina que consume la acción de rodar.
9. `isRunning`: Tipo *Boolean*. Bandera que se levanta cuando el jugador está corriendo.
10. `StaminaRunUse`: Tipo *Float*. Define la cantidad de stamina que gasta el jugador al correr.
11. `runSpeed`: Tipo *Float*. Define la velocidad del jugador al correr.
12. `CurrentSpellIndex`: Tipo *Float*. Índice que indica el hechizo que tiene equipado el jugador actualmente.
13. `SpellImageObject`: Tipo *Image*. Referencia a la imagen que indica el hechizo equipado actualmente en la interfaz.
14. `magicManaUse`: Tipo *Float*. Define la cantidad de maná que consume el proyectil mágico del jugador.
15. `healingManaUse`: Tipo *Float*. Define la cantidad de maná que consume el hechizo de curación.

16. **SwordDamage**: Tipo *Float*. Define el daño base que inflige la espada del jugador.
17. **currentLever**: Tipo *Object Reference* a *BP Lever*. Referencia a la palanca con la que el jugador está interactuando actualmente.
18. **isEnemyDashing**: Tipo *Boolean*. Bandera que indica si el enemigo está realizando una embestida o desplazamiento rápido.
19. **isInCorrectDodgeWindow**: Tipo *Boolean*. Bandera que indica si el jugador se encuentra dentro de la ventana temporal adecuada para ejecutar una esquiva efectiva.
20. **isRollingSideways**: Tipo *Boolean*. Bandera que indica si la esquiva actual se está realizando lateralmente.
21. **MovInput**: Tipo *Vector2D*. Almacena la dirección de movimiento introducida por el jugador.
22. **FireBallImage**: Tipo *Image*. Referencia a la imagen utilizada en la interfaz para representar el hechizo de bola de fuego.
23. **PotionImage**: Tipo *Image*. Referencia a la imagen utilizada en la interfaz para representar las pociones del jugador.
24. **PotiCount**: Tipo *Integer*. Indica la cantidad actual de pociones disponibles.
25. **Potion Count Text Var**: Tipo *Text*. Referencia al elemento de texto de la interfaz que muestra la cantidad de pociones disponibles.
26. **healingPotion**: Tipo *Static Mesh*. Referencia al modelo 3D de la poción de curación utilizada por el jugador.
27. **tmpTarget**: Tipo *Vector*. Almacena una ubicación temporal utilizada por los enemigos como objetivo durante la ejecución de sus comportamientos.

5.3. Sistema de input

La entrada del jugador se gestiona mediante el sistema *Enhanced Input* de Unreal Engine. Cada acción del jugador (atacar, esquivar, correr, etc.) se representa como un *Input Action* independiente, desacoplado de la tecla o botón físico que lo activa. Esta asociación entre la entrada física y el *Input Action* correspondiente se define en un *Input Mapping Context*, lo que facilita el remapeo de controles sin modificar la lógica interna de cada Blueprint. Actualmente el juego solo cuenta con soporte para teclado y mouse.

Los *Input Actions* utilizados por el componente de combate (**BPC_Combat**) y sus controles asociados son los siguientes:

Input Action	Control
IA_Attack	Click izquierdo
IA_Dash-Roll	Barra espaciadora
IA_Run	Shift
IA_Magic	Q
IA_ScrollMagic	Rueda del mouse
IA_Lock	Click rueda del mouse

Input Action	Control
IA_MyInteract	E
IA_EscapeMenu	Escape

En las secciones siguientes, cada acción del jugador se describe haciendo referencia explícita al **Input Action** correspondiente (por ejemplo, **IA_Attack** o **IA_Dash-Roll**), ya activado mediante el *Input Mapping Context* descrito anteriormente.

5.4. Ataque melee del jugador

El ataque del jugador se implementó en el componente de combate mencionado previamente. Al recibirse el **Input Action** **IA_Attack**, antes de ejecutar la acción se comprueba que el jugador no esté ya atacando ni esquivando (variables **isAttacking** e **isDodging**) y, además, que no se esté reproduciendo el *Animation Montage* de reacción a daño (**HitReact_Montage**), de modo que el jugador no pueda atacar mientras recibe un golpe. Si la condición se cumple, se setea **isAttacking** como **true**, se registra el ataque cuerpo a cuerpo en el *Game Instance* (para fines de telemetría) y se ejecuta el *Animation Montage* asociado al ataque, que contiene una secuencia de tres golpes consecutivos.

El encadenamiento de golpes (combo) se controla mediante la variable **isCombo**, que representa la intención del jugador de continuar la secuencia. Esta variable se activa de forma diferida: si el jugador vuelve a recibir el **Input Action** **IA_Attack** mientras un ataque ya está en curso (es decir, cuando la condición inicial es **false** porque **isAttacking** es **true**), en lugar de iniciar un ataque nuevo se setea **isCombo** como **true**. De esta forma el sistema «almacena» la intención de combo hasta el siguiente punto de decisión.

Dicho punto de decisión se evalúa mediante un *Animation Notify* ubicado en el montage, al final del primer y segundo golpe. Cada vez que el notify se activa, se comprueba si el jugador sigue atacando y si tiene la intención de continuar (variables **isAttacking** e **isCombo**). Si ambas son verdaderas, se permite que la secuencia continúe y se resetea **isCombo** a **false** (a la espera de un nuevo input). En caso contrario, se detiene el montage prematuramente mediante **Montage_Stop** y se limpian las variables **isAttacking** e **isCombo**. Cuando el montage se completa o es interrumpido, se restablecen también **isAttacking** e **isCombo** a **false** y se restaura el estado de movimiento del personaje; en el caso de interrupción, se distingue además si la causa fue la reproducción del **HitReact_Montage**.

Con respecto a la detección de impacto del ataque, la implementación se basa en un sistema de *trace* por temporizador. El montage tiene asociado un Blueprint que hereda de *Animation Notify State*, el cual delimita la ventana en la que el arma debe poder golpear mediante un momento de inicio y un momento de fin dentro del montage. Al recibirse la señal de inicio, se invoca el evento **Begin Damage Trace**, que inicia un *timer* en bucle que, cada 0.1 segundos, ejecuta un *Sphere Trace* entre dos componentes de escena anclados al arma (**Start Sword Trace Pos** y **End Sword Trace Pos**), que marcan la base y la punta de

la hoja. Si el *trace* detecta una colisión válida, se llama a la función `ApplyDamage` sobre el actor impactado, utilizando el valor de la variable `SwordDamage` y al jugador como causante del daño. Al recibirse la señal de fin, se invoca el evento `End Damage Trace`, que invalida el *timer*, desactivando la detección hasta el siguiente golpe.

5.5. Projectiles

Tanto el jugador como el jefe pueden lanzar proyectiles. Es por esto que se decidió crear un Blueprint Class que represente la base de un proyectil (`BP_BaseProjectile`), para que después tanto el proyectil del jugador como el del jefe hereden desde esta clase base y ajusten variables específicas de cada uno. El comportamiento por defecto del proyectil es avanzar con velocidad constante en línea recta hasta chocar con algo, momento en el cual reproduce el efecto visual (VFX) y el sonido de impacto correspondientes, aplica daño al objetivo impactado y se destruye.

El proyectil base posee múltiples componentes, entre los cuales están:

1. **BoxCollision:** Componente de colisión (*Box Collision*). Define el volumen de colisión principal del proyectil.
2. **Arrow:** Componente *Arrow*. Sirve como referencia visual y direccional para indicar hacia dónde se moverá u orientará el proyectil.
3. **StaticMesh:** Componente *Static Mesh*. Representa la malla estática visible del proyectil, en caso de necesitarse.
4. **ProjectileMovement:** Componente *Projectile Movement*. Controla el movimiento del proyectil, incluyendo velocidad, gravedad y comportamiento de persecución (*homing*).

Las variables de las que se compone son:

1. **Speed:** Tipo *Float*. Define la velocidad a la que se mueve el proyectil.
2. **Gravity:** Tipo *Float*. Controla cuánta influencia tiene la gravedad sobre el proyectil.
3. **Homing:** Tipo *Boolean*. Indica si el proyectil tiene la capacidad de perseguir automáticamente a un objetivo.
4. **Base Damage:** Tipo *Float*. Define el daño base que causa el proyectil al impactar.
5. **Impact Effect:** Tipo *Particle System*. Es la referencia al sistema de partículas que se reproduce cuando el proyectil impacta.
6. **Sound Impact:** Tipo *Sound Base*. Es la referencia al sonido que se reproduce cuando el proyectil impacta.

El comportamiento del proyectil base se define mediante los siguientes eventos. Al inicializarse (`BeginPlay`), el proyectil configura su componente de colisión para ignorar al actor que lo originó (`GetOwner`), evitando así que colisione consigo mismo o con quien lo disparó. Además, si la variable `Homing` está activada, se habilita el modo de persecución del componente *Projectile Movement* (`bIsHomingProjectile`) y se establece el componente objetivo de la persecución.

La detección de impacto se gestiona mediante un evento ligado al solapamiento del componente de colisión. Cuando el proyectil se solapa con otro actor, se comprueba que dicho actor no sea su propio dueño (`GetOwner`); de no serlo, se invoca la función `SpawnImpactEffect`, que reproduce el sistema de partículas `Impact Effect` en el punto de colisión junto con el sonido `Sound Impact`. A continuación se aplica el daño definido en `Base Damage` sobre el actor impactado —indicando al dueño del proyectil como causante del daño— y finalmente el proyectil se destruye.

5.5.1. Proyectil del jugador

Uno de los hechizos que puede utilizar el jugador es el de disparar un proyectil. El Blueprint que representa al proyectil del jugador (`BP_PlayerFireBall`) hereda de `BP_BaseProjectile` y configura las siguientes variables: `Speed` con un valor alto (para que sea más rápido que el proyectil del jefe), `Gravity` en 0 (sin influencia gravitacional), `Homing` como `false` (no persigue objetivos), y el `Impact Effect` con un VFX de explosión de fuego. En cuanto a los componentes visuales, el `StaticMesh` permanece vacío y en su lugar se utiliza un componente *Particle System* que representa el efecto visual del proyectil en movimiento.

5.5.2. Proyectil del jefe

El jefe puede lanzar proyectiles mediante ciertos ataques. El Blueprint que los representa (`BP_BossSlimeBall`) hereda de `BP_BaseProjectile` y configura sus variables de la siguiente manera: `Speed` con un valor moderado (menor que el del jugador), `Gravity` en 0, el `Impact Effect` con un VFX de explosión de slime, y `Homing` que varía según el tipo de ataque. El jefe dispone de dos variantes de ataque con proyectiles: uno donde el proyectil avanza en línea recta a velocidad constante (`Homing` en `false`), y otro donde persigue activamente al jugador hasta alcanzarlo o explotar tras un tiempo determinado (`Homing` en `true`). Al igual que el proyectil del jugador, el `StaticMesh` permanece vacío y se emplea un componente *Particle System* para el efecto visual del proyectil en movimiento.

Esta subclase sobrescribe (*override*) el evento `BeginPlay` para extender el comportamiento heredado: al instanciarse, primero ejecuta el `BeginPlay` de la clase base (configurando la colisión y el *homing* según corresponda) y, a continuación, inicia un temporizador de 5 segundos. Transcurrido ese tiempo, si el proyectil aún no ha impactado contra nada, spawnear un sistema de partículas Niagara (`NS_Projectile_03_Hit`) en su ubicación actual y se autodestruye, garantizando así que los proyectiles que no alcancen al jugador no permanezcan indefinidamente en la escena.

5.6. Hechizos y consumibles del jugador

El jugador dispone de dos opciones que puede alternar y utilizar mediante una misma acción: un hechizo de proyectil de fuego y una poción de vida. Esta mecánica se implementó con el propósito de otorgar al jugador versatilidad en combate —pudiendo elegir entre una opción ofensiva y una defensiva— y de proporcionar al jefe la oportunidad de demostrar capacidad de aprendizaje y adaptación a las diferentes estrategias del jugador.

El jugador puede intercambiar entre el hechizo de fuego y la poción utilizando la rueda del mouse, y la opción actualmente equipada se muestra representada con un símbolo en la esquina inferior derecha de la pantalla. La selección se representa mediante la variable `CurrentSpellIndex`, que cambia al mover la rueda del mouse (`IA_ScrollMagic`); además de actualizar el valor, se modifica el ícono y la opacidad de la imagen del HUD para reflejar la opción activa.

Cuando el jugador ejecuta la acción de lanzar (`IA_Magic`), el sistema evalúa la variable `CurrentSpellIndex` para determinar qué opción está equipada. Si corresponde al hechizo de fuego, se comprueba primero que se disponga del maná necesario (`magicManaUse` frente a la variable `mana`) antes de invocar el evento del proyectil; si corresponde a la poción, se invoca directamente el evento de curación, cuya disponibilidad depende de la cantidad de pociones que posea el jugador.

5.6.1. Hechizo de proyectil de fuego

El evento del hechizo de proyectil (`ProjMagicEvent`) comienza comprobando que el jugador no esté atacando ni esquivando, y que no se esté reproduciendo la animación de reacción a daño. Si la condición se cumple, registra el ataque a distancia en el *Game Instance* con fines de telemetría, marca al jugador como atacando y reproduce la animación de lanzamiento, que corresponde a un *Animation Montage*. Durante la reproducción se desactiva temporalmente el movimiento del jugador, ya que mientras lanza el proyectil no debe poder desplazarse; el movimiento se restablece cuando la animación finaliza o es interrumpida.

El proyectil en sí se genera mediante un *Animation Notify* anclado al montage: en el instante en que la animación lo indica, se instancia el proyectil del jugador (`BP_PlayerFireBall`) con el jugador asignado como su dueño (*Owner*), de modo que el proyectil no colisione con quien lo dispara. En ese mismo momento se reproduce el sonido de lanzamiento y se realiza el consumo de maná, llamando a la función `IncreaseVal` del componente `BPC_Stats`, pasando como parámetro el valor de la variable `magicManaUse` con signo negativo para representar la disminución.

5.6.2. Poción de vida

La poción de vida constituye la opción defensiva del jugador. No restaura la vida de forma instantánea, sino que requiere que el jugador beba una poción de una cantidad limitada que posee, representada por la variable `PotiCount` y mostrada en el HUD. El evento correspondiente (`HealingMagicEvent`) comienza comprobando que el jugador no esté atacando ni esquivando, que no se esté reproduciendo la animación de reacción a daño y, fundamentalmente, que disponga de al menos una poción (`PotiCount` mayor que cero).

Si la condición se cumple, se registra el momento de curación en el *Game Instance* (junto con la vida actual y máxima del jugador), se decrementa el contador de pociones y se actualiza el HUD, y se desactiva el movimiento mientras se reproduce la animación de beber, que corresponde a un *Animation Montage*. Para reforzar visualmente la acción,

durante la animación se genera y se ancla la malla de la poción a la mano del jugador, la cual se destruye una vez finalizada. La restauración de vida se realiza mediante un *Animation Notify* anclado al montage: en el momento en que la animación muestra al jugador bebiendo, se llama a la función **IncreaseVal** para restaurar la vida del jugador. Al completarse o interrumpirse la animación, se restablece el movimiento del jugador.

5.7. Esquiva

La esquiva del jugador está implementada en el componente **BPC_Combat**. Se creó una función auxiliar (**GetRollMontage**) para determinar qué animación de roll ejecutar, ya que se puede realizar un roll hacia adelante, hacia atrás, o hacia los lados cuando se tiene un objetivo fijado mediante el sistema de *Target Lock*.

La función **GetRollMontage** determina cuál de las cuatro animaciones de roll ejecutar a partir de la dirección de movimiento del jugador. Para ello construye el vector de intención de movimiento en espacio mundo, combinando la entrada de movimiento (**MovInput**) con la rotación de la cámara (la rotación del controlador), y lo normaliza. Luego proyecta ese vector sobre el *Forward Vector* y el *Right Vector* del personaje mediante productos punto, y compara las magnitudes de ambas proyecciones: si la componente hacia adelante/atrás es mayor o igual que la lateral, el roll se considera frontal; en caso contrario, lateral.

Una vez determinado el eje dominante, el signo del producto punto correspondiente define el sentido exacto y, con ello, el *Animation Montage* que se retorna: hacia adelante (**Dodge_F_MontageDAIKO**) o hacia atrás (**Dodge_B_MontageDAIKO**) en el caso frontal, y hacia la derecha (**Dodge_R_MontageDAIKO**) o hacia la izquierda (**Dodge_L_MontageDAIKO**) en el caso lateral. Cuando el roll es lateral, además se marca la variable **isRollingSideWays** como **true**.

Adicionalmente, esta función realiza un registro de telemetría específico para las esquivas realizadas frente a una embestida del jefe: si en el momento del roll el jefe se encuentra embistiendo (**isEnemyDashing**) y el jugador está dentro de una zona determinada (**isInTankZone**), se llama a la función **RegisterDashDodge** del *Game Instance*, pasándole el parámetro **bWasLateral** para indicar si la esquiva fue lateral (**true**, en los rolls hacia los lados) o frontal (**false**, en los rolls hacia adelante o atrás). Esto permite recopilar métricas diferenciadas sobre cómo responde el jugador ante el ataque de embestida según la dirección de su evasión.

En cuanto a la lógica del roll: cuando se recibe el **Input Action IA_Dash-Roll**, primero se comprueba que el jugador no esté atacando (variable **isAttacking**), que disponga de la stamina suficiente para realizar la esquiva (comparando la **stamina** actual contra **StaminaRollUse**) y que no se esté reproduciendo la animación de reacción a daño (**HitReact_Montage**). Si la condición se cumple, se registra la esquiva en el *Game Instance* mediante la función **RegisterTotalDodge** y se procede a ejecutar la acción: se llama a la función **IncreaseVal** para restar la stamina consumida, se marca **isDodging** como **true**, se reproduce el sonido de esquiva y se ejecuta la animación de roll correspondiente (obtenida

mediante `GetRollMontage`). Inmediatamente se setea la variable `canRecoverStamina` como `false` para evitar que se regenere stamina durante la esquivas.

La limpieza de los estados se realiza al término de la animación de roll. Tanto si el montage se completa exitosamente como si es interrumpido, se restablece `canRecoverStamina` a `true` (permitiendo nuevamente la regeneración de stamina) y se limpian las variables `isDodging` e `isRollingSideWays`, devolviendo al jugador a un estado neutro desde el cual puede volver a actuar.

Adicionalmente, el sistema distingue si la esquivas se realizó dentro de una ventana de tiempo considerada como una evasión precisa o «perfecta», mediante la variable `isInCorrectDodgeWindow`. Cuando este es el caso, se efectúa un registro extra en el *Game Instance* mediante la función `RegisterDodgeFromDelayAttack`, lo que permite recopilar métricas sobre las esquivas realizadas con buen *timing* frente a los ataques enemigos.

Cabe mencionar que todo esto se ejecuta dentro de un nodo *Do Once*, el cual, como bien indica su nombre, solo se ejecuta una vez hasta que se resetee. El reseteo se realiza después de 0.45 segundos mediante un nodo *Delay*. Esto permite que el jugador pueda hacer un roll cada 0.45 segundos, pudiendo así encadenar rolls consecutivos para evadir múltiples ataques.

5.8. Fijado de objetivo (Target Lock)

El sistema de fijado de objetivo (*Target Lock*) permite al jugador centrar la cámara sobre un enemigo y mantenerla orientada hacia él durante el combate, facilitando el seguimiento del objetivo mientras se ataca o esquivas. Está implementado en el componente `BPC_Combat` y se reparte entre dos partes: la activación/desactivación del fijado, gestionada por el `Input Action IA_Lock`, y el mantenimiento de la orientación de la cámara, ejecutado en cada fotograma dentro del evento `Tick`.

Al recibirse el `Input Action IA_Lock`, el sistema actúa según si ya existe un objetivo fijado, almacenado en la variable `Target Lock`. Si ya hay un objetivo, se interpreta como una orden de soltarlo: se limpia la referencia, se restauran los parámetros de rotación del personaje a su comportamiento normal (orientación según el movimiento) y se destruye el widget visual del fijado. Si no hay objetivo, se procede a buscar uno: se realiza un *Sphere Trace* hacia adelante desde el personaje (con un radio y un alcance determinados); si dicho trazado detecta un objetivo válido, se almacena en `Target Lock`, se ajustan los parámetros de rotación para que el personaje se oriente hacia el objetivo, y se genera un widget (`BP_TargetLockWidget`) que se ancla al enemigo como indicador visual del fijado.

Una vez fijado el objetivo, su seguimiento se realiza en el evento `Tick`. Mientras exista un objetivo, cada fotograma se calcula la rotación necesaria para que la cámara apunte hacia él y se aplica a la rotación del controlador del jugador. Para obtener el punto exacto al que mirar, el sistema comprueba si el objetivo implementa la interfaz `BPI_Lockable`: si

la implementa, solicita a través de ella un punto de fijado personalizado, lo que permite que ciertos enemigos —como aquellos cuya geometría se deforma o cuyo centro visual no coincide con su origen— definan explícitamente dónde debe apuntar la cámara; si no la implementa, se utiliza directamente la ubicación del objetivo. Además, mientras el fijado está activo, se reposiciona en pantalla el widget de la barra de stamina.

Actualmente, la única clase que implementa la interfaz `BPI_Lockable` es el jefe (`BP_Slime`). Esto se debe a que su animación de ataque de charco (controlada mediante la variable `isPuddle`) desplaza visualmente al personaje hacia abajo —simulando que se hunde o agacha en el charco— sin que dicho desplazamiento se traduzca en un movimiento real del actor en el mundo. Si la cámara utilizara directamente la ubicación del actor (su origen), el punto de fijado quedaría desalineado respecto a la posición visual del jefe durante esta animación. Al implementar `BPI_Lockable`, el jefe puede reportar un punto de fijado ajustado a su posición visual real en cada momento, manteniendo la cámara correctamente orientada incluso cuando la animación no coincide con la posición lógica del actor.

Capítulo 6

Prueba de concepto y evaluación

Capítulo 7

Conclusión y trabajo futuro

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Bibliografía

Anexo A

Anexo

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.